

Агрегация событий

Иногда БЛ, порождающая публикуемые события, достаточно сложна, и в результате её работы может создаваться избыточный поток событий. Часть событий можно было бы объединить в одно, а некоторые являются полными дубликатами друг друга. Для ограничения потока событий принято применять паттерны [debouncing](#) и [throttling](#). Публикация через сервис агрегации событий недоступна в мобильных и десктопных версиях приложения.

Одним из вариантов реализации паттерна debouncing является игнорирование повторных событий в течение какого-либо интервала времени, а паттерна throttling — отложенный вызов события, в котором передаются необходимые данные, вместо множества событий.

Механизм событий позволяет выполнять эти операции, используя **агрегацию**. Она выполняется через **Сервис агрегации событий** и применяется только к **событиям одинакового типа**, у которых совпадают:

- [имя события](#);
- [уровень изоляции](#);
- список сайтов, для которых публикуется событие;
- получатель (см. [Сценарии доставки события](#)).

При разработке прикладного кода следует избегать логики, которая зависит от точной агрегации нескольких событий в заданном интервале агрегации.

Интервал агрегации

Интервал агрегации — это интервал времени, в течение которого для всех событий одинакового типа будет произведена агрегация. Его устанавливают в конфигурации сервиса в параметре `АгрегацияСобытий.ВремяАгрегации` (значение по умолчанию — 1000 мс). Чтобы переопределить значение интервала агрегации для конкретного события, в конфигурации сервиса устанавливают параметр `АгрегацияСобытий.ВремяАгрегацииДляСобытий`. Параметры нужно определять для сервиса агрегации событий (**event-aggregator**), а не для сервиса, публикующего событие.

В качестве значения параметр принимает JSON объект вида:

```
{ "<имя события>": <значение интервала в мс> }
```

Например, для события `test.event` переопределено значение интервала агрегации — 3 секунды, а для события `test.event2` — 5 секунд.

```
АгрегацияСобытий.ВремяАгрегацииДляСобытий = { "test.event": 3000,
"test.event2": 5000 }
```



При агрегации событий нет гарантий четкого соответствия параметрам сервиса **event-aggregator**. Возможно получение нескольких событий (например, 2-3) в заданном интервале агрегации.

Ключ агрегации

Центральным понятием сервиса агрегации событий является [ключ агрегации](#).

Событие будет признано дубликатом, если его ключ агрегации совпадает с одним из ключей, полученных ранее от событий этого же типа в этот же интервал агрегации. Дубликаты событий удаляются.

Событие, не признанное дубликатом, объединяется с событиями этого же типа, которые получены в этот же интервал агрегации. Следствием объединения событий является объединение объектов событий (`message/payload`) в один объект (JSON-массив), который будет отправлен в результирующем событии. Объект последнего объединяемого события будет добавлен последним элементом массива. Например, если телами событий одного типа являются числа 1 и 2, то телом результирующего события станет массив `[1, 2]`.

Сторона, принимающая событие, будет получать массив объектов JSON. При переводе публикации события на использование сервиса агрегации необходимо согласовать этот перевод со стороной, принимающей событие. Если в рамках интервала агрегации сервис получит только одно событие какого-то типа, то объектом результирующего события всё равно будет JSON-массив.

Например, если сервис получил объект с телом "text", то будет опубликовано событие с объектом ["text"].

Гарантии доставки событий

Сервис агрегации событий сохраняет все данные об агрегируемых событиях в оперативной памяти сервера. При аппаратном или программном сбое сервера память будет очищена, а данные о событиях - потеряны. В итоге, клиент не получит сообщения о событиях. Это замечание справедливо для любых способов подписки на события, включая ситуацию, когда подписчик-сервис БЛ установил [durable](#) подписку. В случае штатного выключения сервиса агрегации событий все неотправленные события публикуются независимо от установленного для них значения интервала агрегации. Таким образом, потеря событий при штатном завершении работы сервиса агрегации событий не происходит.



Сервис event-aggregator предназначен для агрегации "массовых" событий и необходим для снижения нагрузки для получателя события. Таким образом, получение двух-трех событий на получателе из 20 опубликованных не является ошибкой работы сервиса.

Пример публикации через сервис агрегации

Пример 1. Абстрактный пример.

- C++

```
1 String key;
2 event::Event ev;
3 ...
4 // Агрегировать только клиентские события.
5 // Объединять тела событий в массив.
6 // Равнозначно ev.SetAggKey( key );
7 ev.SetAggKey( event::AggregationKey( key ) );
8
9 // Агрегировать только клиентские события.
10 // Не объединять тела событий в массив.
11 ev.SetAggKey( event::AggregationKey( key, false, false ) );
12
13 // Агрегировать клиентские и сервисные события.
14 // Объединять тела событий в массив.
15 ev.SetAggKey( event::AggregationKey( key, true ) );
16
17 // Агрегировать клиентские и сервисные события.
18 // Не объединять тела событий в массив.
19 ev.SetAggKey( event::AggregationKey( key, true, false ) );
```

- Python

```
1 # Агрегировать только клиентские события.
2 # Объединять тела событий в массив.
3 # Равнозначно event.Publish( event.Event( ..., agg_key = "key" ) )
4 event.Publish( event.Event( ..., agg_key = event.AggregationKey( key =
  "key" ) ) )
5
6 # Агрегировать только клиентские события.
7 # Не объединять тела событий в массив.
8 event.Publish( event.Event( ..., event.AggregationKey( key = "key",
  use_array = False ) ) )
```

```

9
10 # Агрегировать клиентские и сервисные события.
11 # Объединять тела событий в массив.
12 event.Publish( event.Event( ..., event.AggregationKey( key = "key",
    server = True ) ) )
13
14 # Агрегировать клиентские и сервисные события.
15 # Не объединять тела событий в массив.
16 event.Publish( event.Event( ..., event.AggregationKey( key = "key",
    server = True, use_array = False ) ) )

```

Пример 2. Пользователь производит загрузку папки с множеством вложенных файлов. Каждый файл, с точки зрения сервиса СБИС.Диск, это отдельная сущность. Сервис не распознаёт, что пользователь производит загрузку содержимого какой-то папки. Поэтому после каждого файла сервис СБИС.Диск публикует событие — добавлен новый файл. Чтобы не публиковать событие каждый раз при загрузке файлов, используют объединение событий. Это позволяет уведомлять пользователя о загрузке сразу нескольких файлов. Ключи агрегации для каждого события должны быть различными.

- C++

```

1 #include <sbis-event-subscription/publish.hpp>
2 #include <sbis-event-subscription/event.hpp>
3 ...
4 // Для каждого файла формируется уникальный ключ агрегации.
5 String agg_key = root_name + event_type + file_id;
6 event::Event ev( L"disk.filehasbeenloaded", event::evCLIENT_ONLY );
7 ev.SetPayload( message );
8 ev.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
9 ev.SetUser( user_id );
10 ev.SetAggKey( agg_key );
11 event::Publish( ev );
12 ...

```

- Python

```

1 import event
2 ...
3 # Для каждого файла формируется уникальный ключ агрегации.
4 agg_key = root_name + event_type + file_id
5 event.Publish( event.Event(
6     name="disk.filehasbeenloaded",
7     payload=message,
8     visibility=event.Visibility.evCLIENT_ONLY,
9     policy=sbis.PublicationPolicy.ppIMMEDIATELY,
10    users=[user_id],
11    agg_key=agg_key
12 )
13 )
14 ...

```

Пример 3. При загрузке файла на СБИС.Диск производится запуск процесса перерасчёта размера вверх по иерархии структуры. В этом случае используется удаление дублирующих событий. Ключи агрегации событий должны быть одинаковыми.

- C++

```

1 # include <sbis-event-subscription/publish.hpp>
2 # include <sbis-event-subscription/event.hpp>
3 ...
4 // Для каждой папки формируется одинаковый ключ агрегации.
5 String agg_key = root_name + event_type + folder_id;
6 event::Event ev( L"disk.recalculationhasbeenstarted",
    event::evCLIENT_ONLY );

```

```

7  ev.SetPayload( message );
8  ev.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
9  ev.SetUser( user_id );
10 ev.SetAggKey( agg_key );
11 event::Publish( ev );
12 ...

```

- Python

```

1  import event
2  ...
3  # Для каждой папки формируется одинаковый ключ агрегации.
4  agg_key = root_name + event_type + folder_id
5  event.Publish( event.Event(
6      name="disk.recalculationhasbeenstarted",
7      payload=message,
8      visibility=event.Visibility.evCLIENT_ONLY,
9      policy=sbis.PublicationPolicy.ppIMMEDIATELY,
10     users=[user_id], agg_key=agg_key
11 )
12 )
13 ...

```